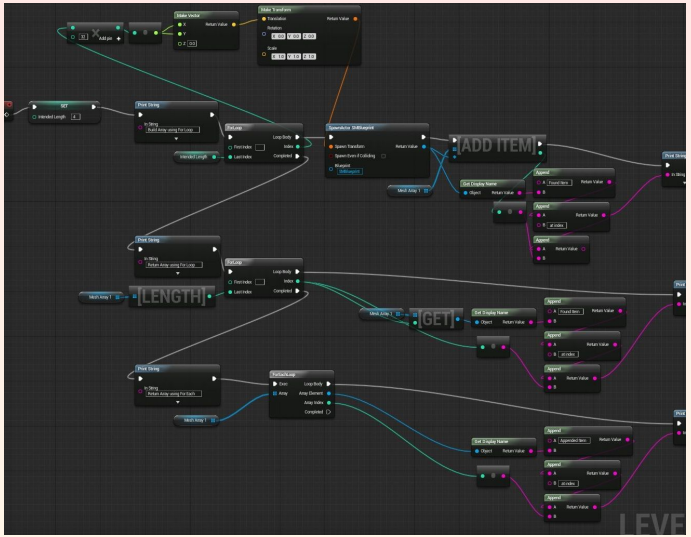
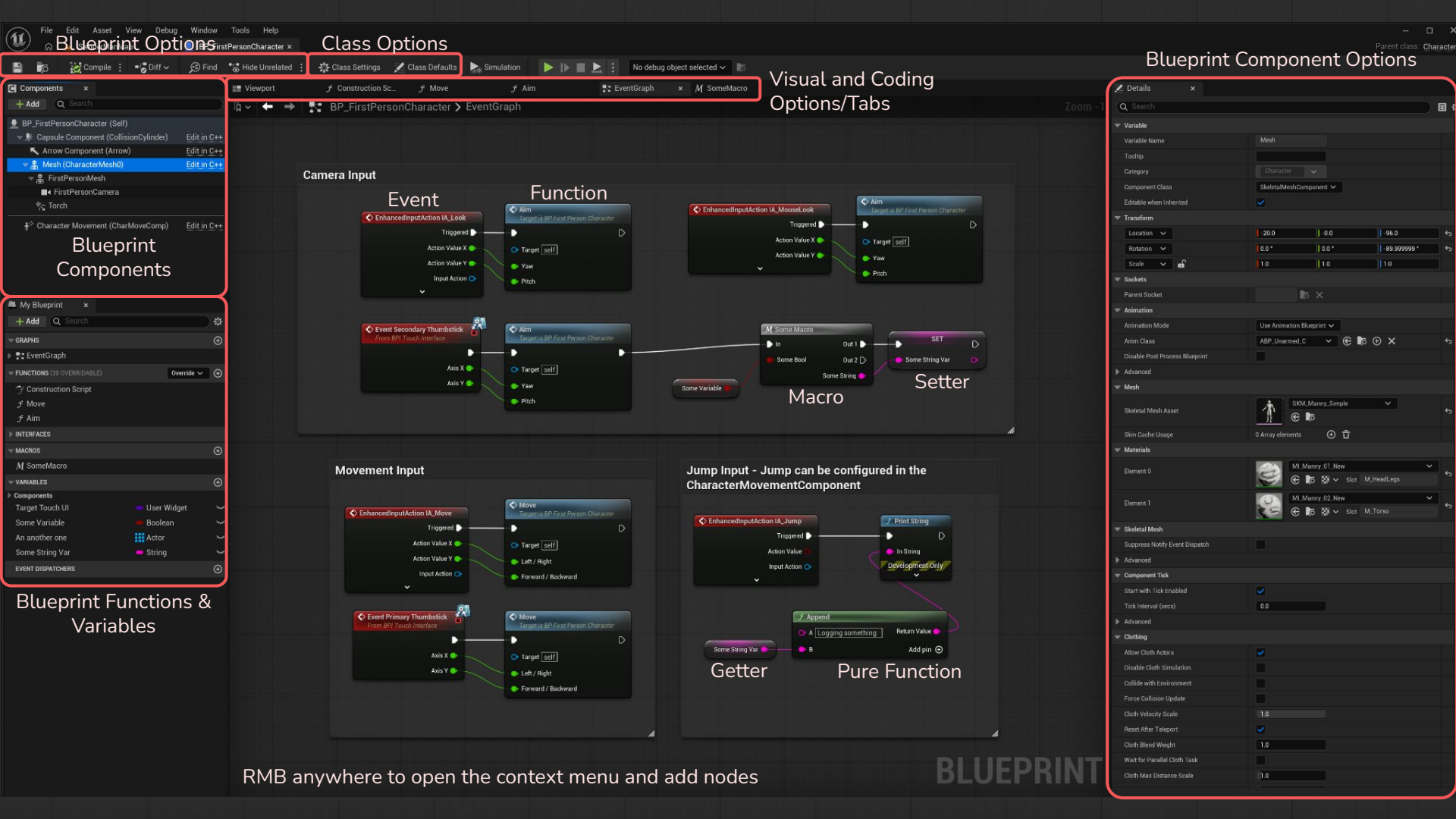
Game Engine Architectures 02

Blueprints && Programming Basics

What are Blueprint Classes?

- Node-based Visual Programming
  (connect nodes to create code)
- Can only be reached from Code when a C++ class is set as parent
- Accessible to Programmers and Designers
- Don’t have to contain scripted behaviour; similar to Unity Prefabs
- Version Control difficult: BPs are saved as binary code
- Documentation




Blueprint Options
Class Options
Blueprint Component Options
Visual and Coding Options/Tabs
Blueprint Components
Blueprint Functions & Variables
Camera Input
Event
Function
Setter
Macro
Movement Input
Jump Input - Jump can be configured in the CharacterMovementComponent
Getter
Pure Function
RMB anywhere to open the context menu and add nodes
BLUEPRINT

Event Graph vs. Construction Script

Event Graph

- the typical coding experience
- uses events and function calls to perform actions in response to gameplay events
- can access all components of the BP and do things with it

Examples:

- when pressing a button, toggle a light
- a bullet getting destroyed when hitting a wall

Construction Script

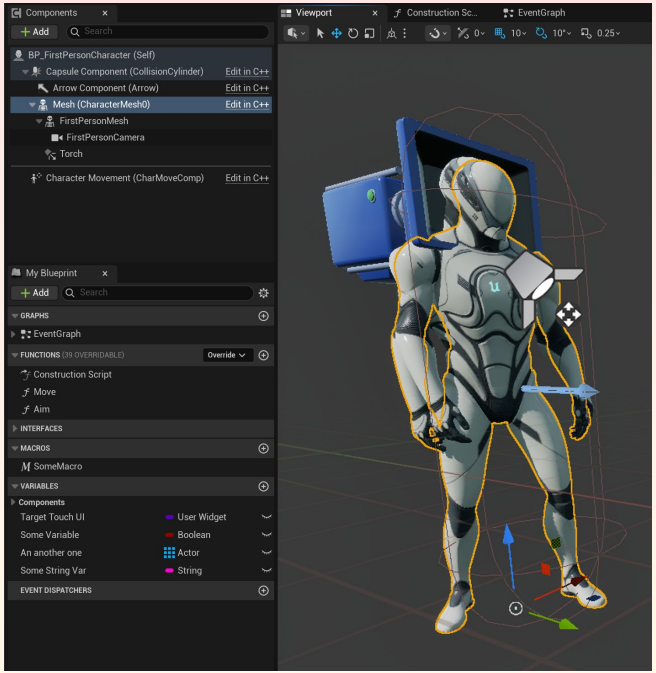
- runs when an instance of a BP is created
- also runs in Edit Mode!
- often used to dynamically adjust the look and feel of actors

Examples:

- set up the Tiles and Pieces of a chess board
- randomise Tree Variant every time it's moved

Viewport

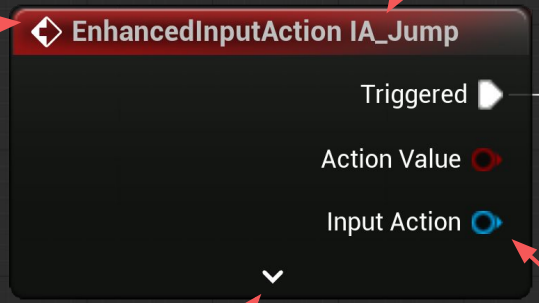
- Inside your Blueprint Class
- The visual part of your BP: this is where you see how your Actor will look like when placed in the level
- You can navigate inside it like in a level, select components, transform them, change settings etc.
- Made up of *Actor Components*, *Scene Components* and *Primitive Components* (more to that next lesson)

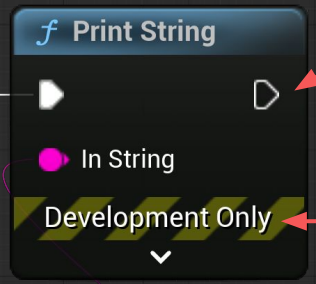


Anatomy of a Node

the name can change depending on some variables you set in the node

Colour and Icon show type of node



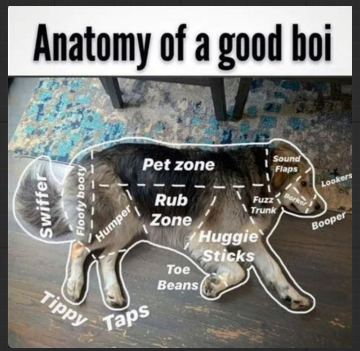


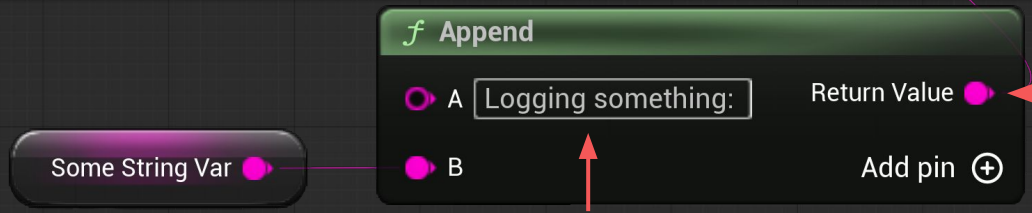
Exec Pin:
when the game is running, it will follow the exec line; everything not connected to it will not be called

will not execute in builds

sometimes there's more hidden here

right click a Pin for more options
(e.g. promote to variable or split structs)



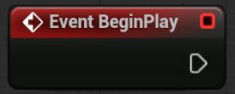


Data Pin
(colour coded)
(input vs. output pins)

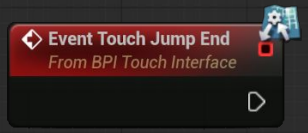
you can also hard-code values sometimes

Node Examples

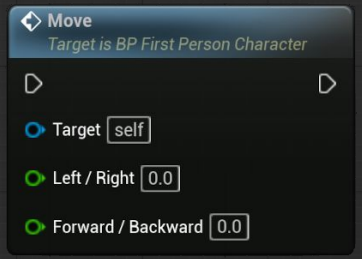
Event



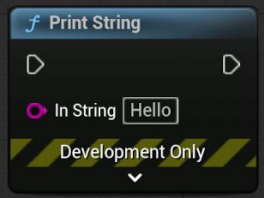
Interface Event
(sometimes there's icons in the top right that explain the type of node in more detail)

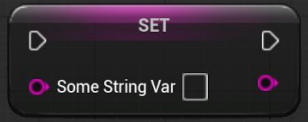


Function

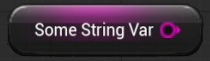


Function
(can be C++ or BP based)

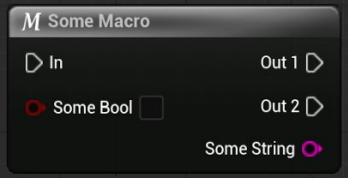




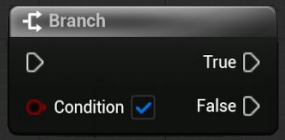
Setter



Getter

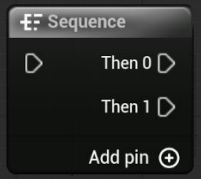


Macro
(like a collection of nodes; not the same as functions - they don't have their own scope)



Nodes from UE Library

Tip: hold "B" and click into graph to create Branch Node



Nodes from UE Library

Tip: hold "S" and click into graph to create Sequence Node

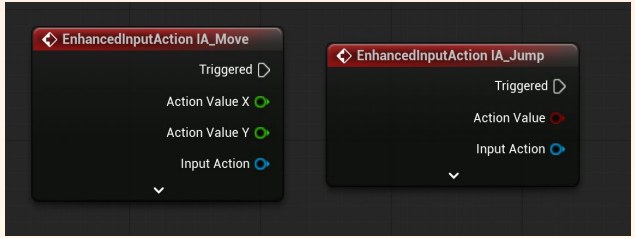
Input in UE5

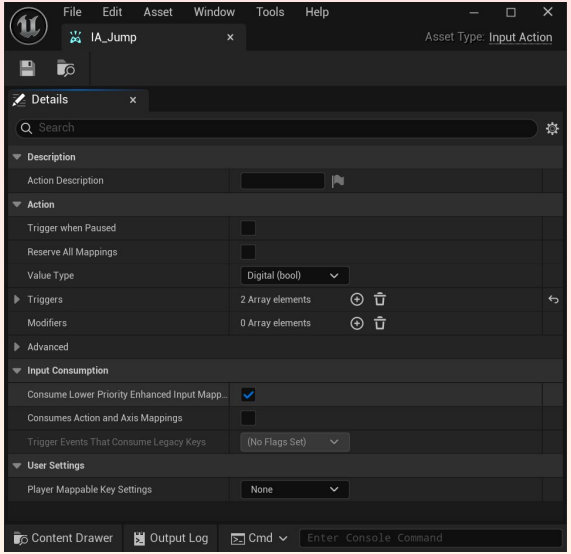
Enhanced Input System (EIS) #1

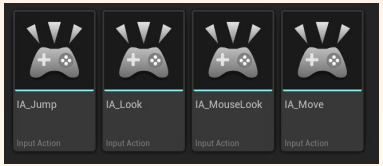
Four main concepts

Input Actions

- link between EIS and Code
- defines *how* an action is triggered
- should represent something the user can do (e.g. “Crouch”)
- Trigger States (triggered, started, ongoing, completed, canceled)





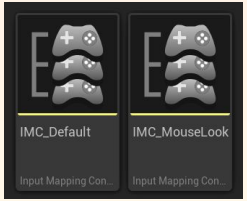


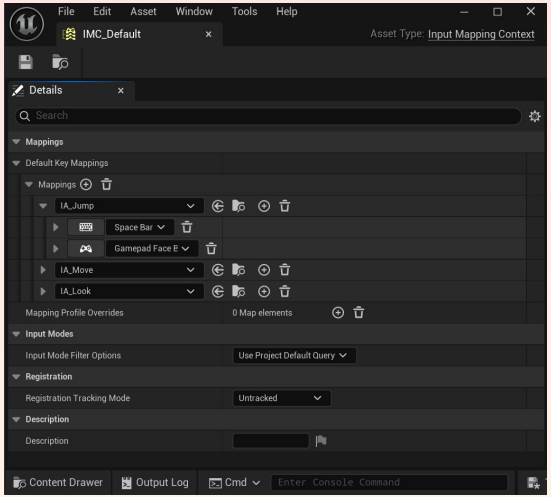
Enhanced Input System (EIS) #2

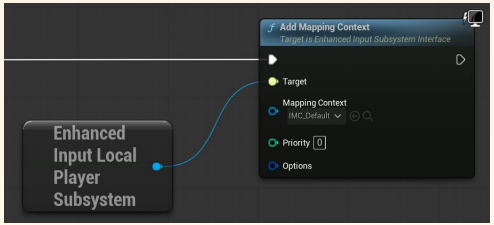
Four main concepts

Input Mapping Contexts

- a collection of Input Actions
- represent a certain context the player can be in
- set rules for what triggers a specific Input Action
- this is what is passed to the EIS and can be changed on runtime







Enhanced Input System (EIS) #3

Four main concepts

Input Modifiers

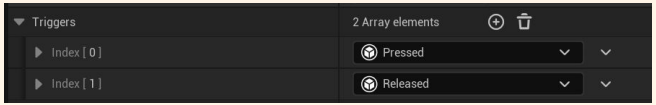
- alter raw input before sending them on
- e.g. to add deadzones, invert axes, etc.
- added in Input Action
- you can even create your own!



Four main concepts

Input Triggers

- gets evaluated after the Input Modifiers
- determine whether user input should activate the corresponding Input Action within its Context
- checks patterns (short taps, hold etc.)



Collision and Physics

(These slides are too bleak but I don't have fitting memes, so have a photo of my cat instead)



Collision vs Trigger Recap

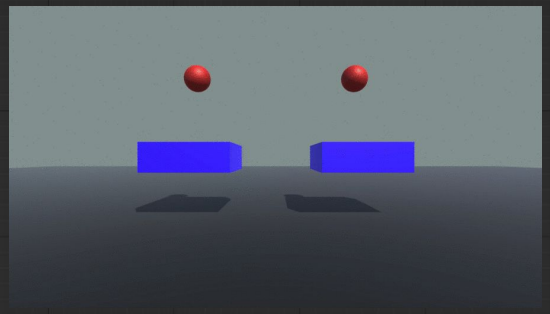
Collision:

- Objects block each other
- Walls, floor, ball bounce etc.

Trigger:

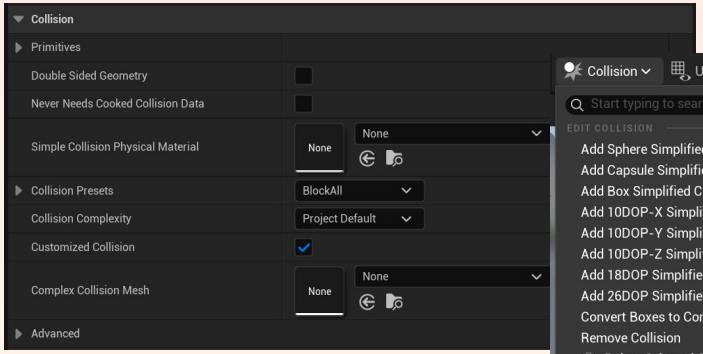
- Objects pass through
- activation zones, pickups etc.

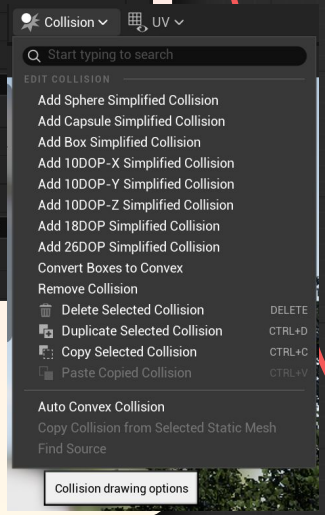
usually the same in every game engine :D



Creating Colliders for Static Meshes

Open the Static Mesh Editor to set your colliders





there's a lot of options to choose from

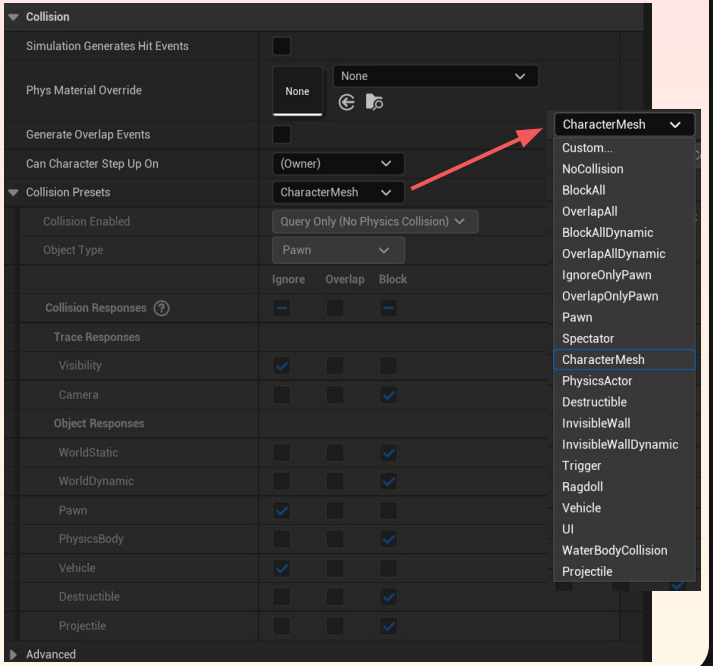


you can add as many colliders and transform them as needed

often simple colliders are enough (and also better for performance!)

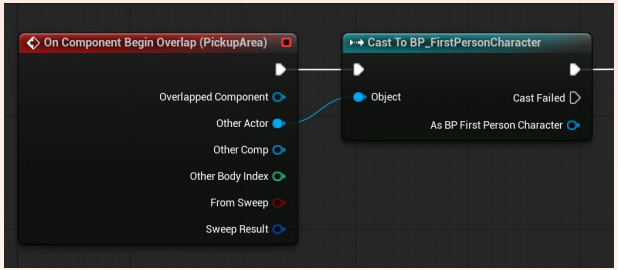
Collision Presets

- There are different predefined modes to handle collision detection between objects
- This can be very difficult to set correctly
- You can also create a custom one and tweak the values yourself
- Most often, using one of the Presets is enough, though
- Documentation
- Explanation of Presets



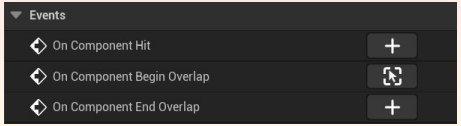
Collision events within Blueprints

- When clicking on a component, in the Details panel scroll all the way to the bottom to find events for this component

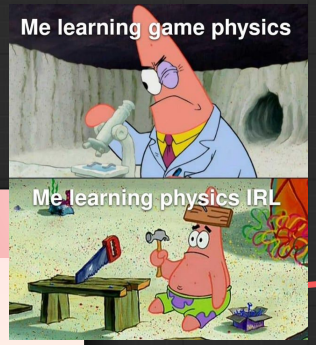


don’t forget to set the Collision Preset to “Trigger” if you want Trigger detection!

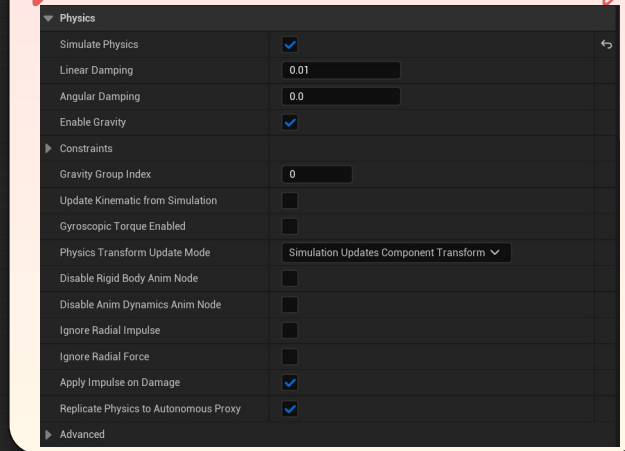
- “OnComponentBeginOverlap” and “OnComponentEndOverlap” are often used for Triggers
- “OnComponentHit” can be used for Collisions



Physics



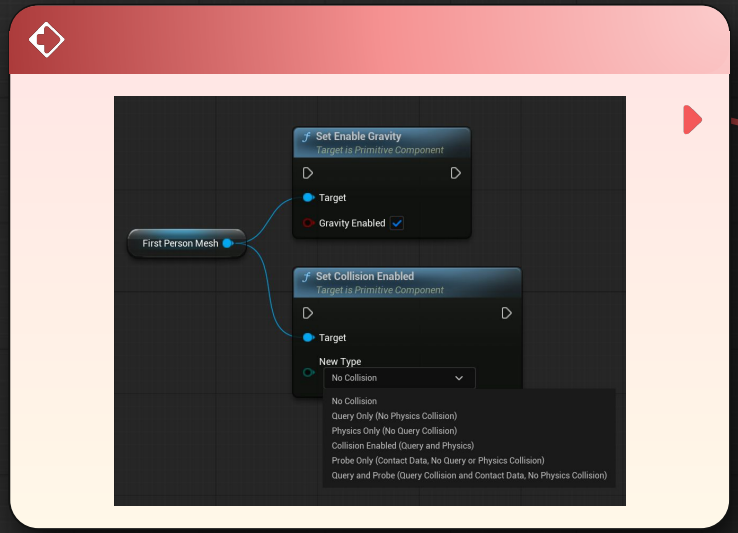
Can be enabled by ticking one box (“**Simulate Physics**”); then calculates everything physics-related on its own



- there are a few settings that you may recognize from Unity’s *Rigidbody* component, like *damping* and *constraints*
- you can access and modify all these settings also in your code

just click on any static mesh component in the component window

Accessing within Blueprints



Query: only for spatial queries (raycasts, sweeps, and overlaps); can't be used for simulation (rigid body, constraints)

Physics: only for physics simulation (rigid body, constraints); can't be used for spatial queries (raycasts, sweeps, overlaps)

changing the Collision Preset

